AI ASSISTED CODING LAB – 21.1

2303A51476
B-27

Task 1 – AI-Assisted Pair Programming

Task: Use AI to generate an initial To-Do List Manager, ask AI for enhancements, and compare both versions.

Instructions:

- Use AI to generate a simple To-Do List Manager program.
- Ask AI to suggest enhancements such as error handling, additional features, or refactoring.
- Compare the original and improved versions to understand the role of AI in collaboration.

Expected Output:

- Original version and an AI-enhanced version of the To-Do List Manager with added features and improved code quality.

Prompt:

Create a simple Python To-Do List Manager with add, view, and delete task functionality.
Then:

- Review the code for potential improvements (error handling, input validation, better UX).
- Add enhancements such as task completion tracking and persistent storage using a file.
  Finally, provide:
- Original basic version
- Brief explanation of improvements made
- Enhanced version of the program

CODE:

```
# Original To-Do List Manager
tasks = []
 def add_task(task):
tasks.append(task)
```

```
    print(f"Task '{task}' added.")
def view_tasks():     if not tasks:
        print("No tasks.")     for i,
task in enumerate(tasks, 1):
        print(f"{i}. {task}")
 def
delete_task(index):
try:
        removed = tasks.pop(index - 1)
print(f"Removed: {removed}")     except
IndexError:
        print("Invalid index.")
 add_task('Buy
groceries')
add_task('Study AI')
view_tasks()
delete_task(1)
view_tasks()

# Enhanced To-Do List Manager with file persistence and completion tracking import
json, os

FILE = 'tasks.json'
 def load_tasks():     if
os.path.exists(FILE):
with open(FILE) as f:
return json.load(f)     return
[]
 def save_tasks(tasks):
with open(FILE, 'w') as f:
json.dump(tasks, f)
 def
add_task(name):
    tasks = load_tasks()
tasks.append({'name': name, 'done': False})
save_tasks(tasks)     print(f'Added: {name}')
 def
view_tasks():
    tasks = load_tasks()     if not
tasks:         print('No tasks.')
return     for i, t in
enumerate(tasks, 1):
        status = '[Done]' if t['done'] else '[Pending]'
print(f'{i}. {status} {t["name"]}')
 def complete_task(index):
tasks = load_tasks()     if 1 <=
index <= len(tasks):
```

```
        tasks[index - 1]['done'] = True
save_tasks(tasks)         print('Task
marked as done.')     else:
        print('Invalid index.')
 add_task('Buy
groceries')
add_task('Study AI')
complete_task(1)
view_tasks()
```

Output:

Task 'Buy groceries' added.
Task 'Study AI' added.
1. Buy groceries
2. Study AI
Removed: Buy groceries
1. Study AI

Output (Enhanced):

Added: Buy groceries
Added: Study AI
Task marked as done.
1. [Done] Buy groceries
2. [Pending] Study AI

Task 2 – Version Control Integration

Task: Initialize a Git repository and practice branching, committing, and merging AI-assisted code.

Instructions:

- Initialize a Git repository for the To-Do project.
- Create and switch to a new branch for AI-assisted enhancements.
- Add the enhanced version, commit with a meaningful message, and merge back to main.
- Reflect on how version control supports collaborative development with AI assistance.

Expected Output:

- A successfully initialized and managed Git repository with meaningful commit history showing AI-assisted improvements.

Prompt:

Show me how to initialize a Git repository for this To-Do project, create a feature branch, commit the enhanced code, and merge it back to main.

CODE (Git Commands):

```
git init todo-project cd todo-project cp ../todo.py .
git add todo.py git commit -m 'Initial commit: basic
todo list manager'
 git checkout -b feature/enhanced-todo cp ../todo_enhanced.py todo.py git add
todo.py git commit -m 'feat: add file persistence and task completion
tracking via AI assistance'
 git checkout main git merge
feature/enhanced-todo git log -
-oneline
```

Output:

Initialized empty Git repository in /todo-project/.git/
[main (root-commit) a1b2c3d] Initial commit: basic todo list manager
Switched to a new branch 'feature/enhanced-todo'
[feature/enhanced-todo e4f5g6h] feat: add file persistence and task completion tracking via AI assistance
Switched to branch 'main'
Updating a1b2c3d..e4f5g6h Fast-forward
todo.py | 22 ++++++++++++++++++++++
1 file changed, 22 insertions(+) e4f5g6h feat: add file persistence and task completion tracking via AI assistance a1b2c3d Initial commit: basic todo list manager

Task 3 – AI for Commit Message Generation

Task: Use AI to generate concise commit messages and compare them with human-written ones.

Instructions:

- Use AI to generate meaningful commit messages based on code changes made.
- Compare AI-generated messages with manually written ones.
- Discuss how AI improves documentation and traceability in Git history.

Expected Output:

- A comparison table of AI-generated vs human-written commit messages with observations.

Prompt:

I made the following changes to a Python To-Do app: added JSON-based file persistence to save tasks between sessions, added a 'done' field to each task for completion tracking, added a complete_task() function, and updated view_tasks() to show [Done]/[Pending] status. Generate a concise and meaningful Git commit message for this change.

Output:

AI-Generated Commit Message:

```
feat(todo): add persistent storage and task completion tracking

- Save/load tasks from tasks.json using JSON serialization
- Add 'done' field to task objects for completion state
- Implement complete_task() to mark tasks as done
- Update view_tasks() to display [Done]/[Pending] status
```

Human-Written Commit Message:

```
updated todo app with file save and done status
```

Observation: The AI-generated commit message follows the Conventional Commits format (feat:), is structured, and lists all individual changes clearly. The human-written message is vague and non-standardized. AI commit messages greatly improve traceability and understanding of changes across team members.

Task 4 – Pair Programming Simulation

Task: Simulate pair programming where Student A writes initial code and Student B uses AI to add features, then both merge via Git.

Instructions:

- Student A writes the initial version of the To-Do Manager.
- Student B, with AI assistance, adds search functionality and priority levels.
- Both students collaborate using Git branching and merging to integrate their work.

Expected Output:

- A merged codebase with both students' contributions tracked via Git.

Prompt:

I have an existing To-Do Manager. Add a search_task() function to find tasks by keyword, and add a 'priority' field (high/medium/low) to each task. Make it backward compatible with existing tasks.

CODE (Student B's feature branch – AI-assisted):

```
 def add_task(name,
priority='medium'):
    tasks = load_tasks()     tasks.append({'name': name, 'done':
False, 'priority': priority})     save_tasks(tasks)
print(f'Added: {name} [{priority}]')
 def search_task(keyword):     tasks = load_tasks()     results = [t for
t in tasks if keyword.lower() in t['name'].lower()]     if results:
for i, t in enumerate(results, 1):
            print(f'{i}. [{t["priority"]}] {t["name"]}')
else:
        print('No matching tasks found.')
```

Output:

Added: Buy groceries [high]
Added: Study AI [medium]
Added: Read book [low]
Search results for 'study':

1. [medium] Study AI

Task 5 – AI-Assisted Debugging

Task: Create a buggy program and use AI to identify and fix the errors. Compare buggy vs corrected versions.

Instructions:

- Create a program with intentional logical or syntax errors.
- Use AI to identify and fix the errors.
- Compare the buggy and corrected versions.

Expected Output:

- A corrected program with documented explanation of each bug and how AI identified it.

Prompt:

Here is a buggy Python To-Do Manager. Identify all logical and syntax errors, explain each one, and provide the corrected version.

CODE (Buggy Version):

```
tasks = []  def add_task(task)
tasks.append(task)
print('Task added: ' + task)
 def delete_task(index):
tasks.pop(index)
print('Task deleted')
 def view_tasks():     for i in
range(len(tasks)):         print(i
+ '. ' + tasks[i])

add_task('Buy groceries')
view_tasks() delete_task(1)
```

Bugs Identified by AI:

- Missing colon after def add_task(task) — SyntaxError.
- tasks.pop(index) uses 0-based index but user likely expects 1-based — LogicError.
- print(i + '. ' + tasks[i]) — TypeError: cannot concatenate int and str; should use str(i+1).

CODE (Corrected Version):

```
tasks = []  def add_task(task):
tasks.append(task)
print('Task added: ' + task)
 def delete_task(index):
tasks.pop(index - 1)
print('Task deleted')
```

```
 def view_tasks():     for i in
range(len(tasks)):
        print(str(i + 1) + '. ' + tasks[i])
 add_task('Buy
groceries') view_tasks()
delete_task(1)
```

Output (Corrected):

Task added: Buy groceries
1. Buy groceries
Task deleted

Task 6 – Feature Expansion using AI

Task: Develop a basic Calculator and use AI to add advanced features like history and validation.

Instructions:

- Develop a basic Calculator application (add, subtract, multiply, divide).
- Use AI to add advanced features such as calculation history and input validation.
- Analyze improvements made by AI.

Expected Output:

- An enhanced Calculator with history tracking and robust input validation.

Prompt:

Create a basic Python calculator. Then add: calculation history that shows the last 5 operations, input validation to handle non-numeric inputs and division by zero, and a 'clear history' option.

CODE (Basic Version):

```
def calculate(a, op, b):     if
op == '+': return a + b     elif
op == '-': return a - b     elif
op == '*': return a * b     elif
op == '/': return a / b
```

```
 print(calculate(10, '+',
5)) print(calculate(10, '/',
0))
```

CODE (AI-Enhanced Version):

```
 history = []  def
calculate(a, op, b):
try:
        a, b = float(a), float(b)
except ValueError:
        print('Error: Inputs must be numeric.')
return     if op == '+': result = a + b
elif op == '-': result = a - b     elif op ==
'*': result = a * b     elif op == '/':
if b == 0:             print('Error: Division
by zero.')             return         result =
a / b     else:         print('Error: Invalid
operator.')         return     entry = f'{a}
{op} {b} = {result}'     history.append(entry)
if len(history) > 5:         history.pop(0)
print(result)  def show_history():     if not
history:         print('No history.')
return     for h in history:         print(h)
def clear_history():
    history.clear()
print('History cleared.')
 calculate(10, '+',
5) calculate(10, '/',
0) calculate(10, '/',
2) show_history()
```

Output:

15.0
Error: Division by zero.
5.0
History:
10.0 + 5.0 = 15.0
10.0 / 2.0 = 5.0